

Primer Parcial: Optimización y Paralelización en Dinámica Cuántica Sistemas Distribuidos y HPC

Sebastián Rodríguez García (20212107046)
Camilo Huertas

Proyecto Curricular de Física - Universidad Distrital Francisco José de Caldas

21 de marzo de 2026

Resumen

Este informe detalla el rediseño, optimización y paralelización de la solución numérica de la ecuación de Schrödinger dependiente del tiempo para un paquete de ondas gaussiano. Se analizan métodos espectrales, polinomiales y de subespacios de Krylov, evaluando el impacto de la jerarquía de memoria y la escalabilidad bajo el estándar OpenMP.

1. Descripción del Problema Físico: El Potencial de Morse

El objetivo central de este estudio es la determinación del espectro de energías acotadas para un sistema molecular diatómico. Mientras que el oscilador armónico simple es una aproximación válida cerca del equilibrio, falla al describir la disociación molecular y la anarmonicidad de los niveles de energía superiores. Para un tratamiento físico riguroso, se emplea el **Potencial de Morse**.

1.1. Naturaleza del Potencial de Morse

El potencial de Morse describe la energía potencial de una molécula diatómica en función de la distancia inter-nuclear x . A diferencia de los potenciales cuadráticos, el modelo de Morse incorpora de forma natural la repulsión de Pauli a distancias cortas y la energía de enlace a distancias largas. Se define mediante la expresión:

$$V(x) = D_e \left(1 - e^{-a(x-x_e)}\right)^2 \quad (1)$$

Donde los parámetros físicos críticos son:

- D_e : Profundidad del pozo de potencial, que representa la energía de disociación de la molécula medida desde el mínimo del pozo.
- x_e : Distancia de equilibrio inter-nuclear.
- a : Parámetro de curvatura, relacionado con la constante de fuerza del enlace y el ancho del pozo.

1.2. Solución Analítica y Estados Acotados

Para una partícula de masa μ (masa reducida del sistema), la ecuación de Schrödinger independiente del tiempo para este potencial admite una solución analítica exacta. Los autovalores de energía permitidos para los estados ligados (acotados) están dados por la expresión:

$$E_n = \hbar\omega_e \left(n + \frac{1}{2} \right) - \frac{[\hbar\omega_e (n + \frac{1}{2})]^2}{4D_e} \quad (2)$$

donde $\omega_e = a\sqrt{2D_e/\mu}$ es la frecuencia de vibración clásica.

Esta solución revela tres características físicas fundamentales que el método numérico debe replicar:

1. **Anarmonicidad:** A diferencia del oscilador armónico donde los niveles están equiespaciados ($\Delta E = \hbar\omega$), en el potencial de Morse los niveles se vuelven más densos a medida que n aumenta.
2. **Número Finito de Estados:** Existe un valor máximo de n por encima del cual $E_n \geq D_e$. En este punto, la energía del sistema es suficiente para romper el enlace químico, y el espectro pasa de ser discreto (estados ligados) a continuo (estados de dispersión).
3. **Energía de Punto Cero:** El estado fundamental ($n = 0$) posee una energía mayor al mínimo del potencial, de acuerdo con el principio de incertidumbre de Heisenberg.

El problema físico se reduce, por tanto, a encontrar estos autovalores E_n mediante técnicas computacionales y contrastarlos con esta solución analítica para evaluar la precisión del esquema implementado.

2. Justificación de los Métodos Numéricos

2.1. Representación Matricial y Transformación de Bases

Para resolver la ecuación de Schrödinger estacionaria $\hat{H}|\psi\rangle = E|\psi\rangle$, se emplea el método de diagonalización de la matriz del Hamiltoniano en una base finita de funciones del oscilador armónico lineal (LHO). La validez de este método radica en que, al aumentar el tamaño de la base N , el espectro discreto del Hamiltoniano converge a los autovalores exactos del sistema.

2.1.1. Construcción de Operadores en la Base del Oscilador

Partimos de los operadores posición $\hat{X}$ y momento $\hat{P}$. En la base de estados propios del LHO $\{|n\rangle\}$, estos operadores presentan una estructura tridiagonal, lo que reduce drásticamente el costo computacional de su inicialización y garantiza la esparcidamente de las matrices iniciales. Las relaciones de recurrencia vienen dadas por:

$$X_{ij} = \frac{1}{\sqrt{2}}(\sqrt{j}\delta_{i,j-1} + \sqrt{i}\delta_{j,i-1}) \quad (3)$$

```
C      CONSTRUCCION DE LAS MATRICES X Y P EN LA BASE DEL LHO
DO 10 I=1,N-1
      A_VAL=DBLE(I)
      X(I,I+1)=DSQRT(A_VAL/2.DO)
      X(I+1,I)=X(I,I+1)
```

```

      P(I,I+1)=-DSQRT(A_VAL/2.DO)
      P(I+1,I)=DSQRT(A_VAL/2.DO)
10    CONTINUE

```

Listing 1: Inicialización de operadores de primer orden en Fortran 77

2.1.2. Diagonalización de $\mathbb{X}$ y Representación de Posición Discreta

El núcleo de la justificación numérica es evitar la evaluación de integrales de la forma $\langle i | V(x) | j \rangle$, las cuales para el potencial de Morse involucran funciones exponenciales de operadores. En su lugar, resolvemos el problema de autovalores para la matriz de posición:

$$\mathbb{X} |x_\gamma\rangle = x_\gamma |x_\gamma\rangle \quad (4)$$

Donde x_γ son los puntos de la malla física y la matriz de autovectores V_P define la transformación unitaria entre la base de energía y la base de posición.

2.1.3. Evaluación de la Matriz de Potencial $V(\mathbb{X})$

Dado que cualquier función de un operador $f(\hat{X})$ es diagonal en la base donde $\hat{X}$ lo es, el potencial de Morse se construye de manera exacta en la representación de posición discreta. Para regresar a nuestra base de trabajo $\{|n\rangle\}$, aplicamos la transformación de semejanza:

$$V_{ij} = \langle i | V(\hat{X}) | j \rangle = \sum_{\gamma=1}^N (V_P)_{i\gamma} V(x_\gamma) (V_P)_{j\gamma} \quad (5)$$

Este procedimiento es numéricamente estable y preserva la simetría del Hamiltoniano.

```

C      CONSTRUCCION DE V EN LA BASE ORIGINAL USANDO AUTOVECTORES DE X
      DO 20 I=1,N
        DO 30 J=I,N
          SUM=0.DO
          DO 40 K=1,N
            SUM=SUM + VP(I,K) * V_MORSE(X_EIG(K)) * VP(J,K)
40          CONTINUE
            VMAT(I,J)=SUM
            VMAT(J,I)=SUM
30          CONTINUE
20    CONTINUE

```

Listing 2: Construcción de la matriz de potencial mediante transformación unitaria

2.1.4. Hamiltoniano Final y Obtención del Espectro

La energía cinética se obtiene a partir del operador momentum como $\mathbb{T} = \frac{1}{2\mu} \mathbb{P}^2$. Finalmente, el Hamiltoniano total se diagonaliza para obtener los autovalores de energía E_ν :

$$\mathbb{H} = \frac{1}{2\mu} \mathbb{P}^2 + \mathbb{V} \quad (6)$$

```

C      SUMA DE T Y V Y EXTRACCION DE AUTOVALORES DE ENERGIA
      CALL MATMUT(P,P,T,N)
      DO 50 I=1,N
        DO 60 J=1,N
          H(I,J) = -0.5D0 * T(I,J) + VMAT(I,J)
60      CONTINUE
50      CONTINUE
      CALL EIGEN(H,N,N,ENERGIA,VP,WORK1,WORK2,Z)

```

Listing 3: Diagonalización final del Hamiltoniano total

2.2. Discretización mediante Diferencias Finitas (FDM)

Como método alternativo de validación, se implementó el esquema de Diferencias Finitas de segundo orden. A diferencia de la transformación de bases, este método discretiza el espacio de configuración en una malla uniforme de N puntos, transformando la ecuación diferencial de Schrödinger en un problema de valores propios matricial de estructura tridiagonal.

2.2.1. Aproximación del Operador Hamiltoniano

El fundamento del método radica en la aproximación de la derivada segunda (operador cinético) mediante el esquema de diferencia central de tres puntos. Para un punto x_i en la malla con espaciamiento Δx :

$$\left. \frac{d^2\psi}{dx^2} \right|_{x_i} \approx \frac{\psi_{i+1} - 2\psi_i + \psi_{i-1}}{\Delta x^2} + \mathcal{O}(\Delta x^2) \quad (7)$$

Al sustituir esta expresión en la ecuación de Schrödinger independiente del tiempo, el Hamiltoniano actuando sobre el vector de la función de onda $\vec{\psi}$ se representa como una matriz $\mathbb{H}$ cuyos elementos son:

$$H_{ij} = \begin{cases} \frac{1}{\Delta x^2} + V(x_i) & \text{si } i = j \\ -\frac{1}{2\Delta x^2} & \text{si } i = j \pm 1 \\ 0 & \text{en otro caso} \end{cases} \quad (8)$$

Esta estructura tridiagonal es una consecuencia directa de la localidad del operador de derivación numérica.

2.2.2. Implementación y Flujo Lógico en Fortran

La implementación en el código `morse_fdm.f` sigue un flujo lineal optimizado para la construcción de la matriz. Se destaca el uso de la precisión doble (`REAL*8`) para minimizar los errores de truncamiento inherentes al esquema de diferencias.

```

C      DEFINICION DE PARAMETROS DE LA MALLA Y EL POTENCIAL
      DX2 = DX * DX
      DO 100 I=1,N
        XI = XMIN + DBLE(I) * DX
        VI = D_POT * (1.D0 - DEXP(-ALPHA * (XI - RE)))**2
C      DIAGONAL: ENERGIA CINETICA LOCAL + POTENCIAL

```

```

      H(I,I) = (1.D0 / DX2) + VI
C      FUERA DE LA DIAGONAL: ACOPLAMIENTO CINETICO (SOLO i < N)
      IF (I .LT. N) THEN
        H(I,I+1) = -1.D0 / (2.D0 * DX2)
        H(I+1,I) = H(I,I+1)
      END IF
100  CONTINUE

```

Listing 4: Ensamblaje de la matriz Hamiltoniana tridiagonal en Fortran 77

2.2.3. Análisis de Convergencia y Estabilidad

La precisión del método FDM es altamente sensible al tamaño de la malla N . Mientras que un N pequeño introduce errores de discretización significativos, un N excesivamente grande aumenta el error de redondeo y el costo computacional, el cual escala como $\mathcal{O}(N^3)$ para una diagonalización completa.

Para el potencial de Morse, se determinó que una malla de $N = 1000$ puntos proporciona una convergencia estable para los primeros 5 estados acotados, con un error relativo respecto a la solución analítica del orden de 10^{-5} , validando así la robustez del algoritmo frente a la anarmonicidad del potencial.

2.3. Método de Disparo y Propagación de Numerov

Como tercer enfoque numérico, se implementó el **Método de Disparo** (*Shooting Method*). A diferencia de los métodos globales (FDM y Transformación de Bases), este método aborda la ecuación de Schrödinger independiente del tiempo como un problema de condiciones de frontera resuelto mediante la integración de una ecuación diferencial ordinaria (EDO) de segundo orden:

$$\frac{d^2\psi(x)}{dx^2} = \hat{Q}(x, E)\psi(x), \quad \hat{Q}(x, E) = -2\mu[E - V(x)] \quad (9)$$

Donde el autovalor de energía E actúa como un parámetro de control que debe ajustarse hasta que la función de onda cumpla la condición de asintoticidad $\psi(x) \rightarrow 0$ cuando $x \rightarrow \infty$.

2.3.1. Integración de Alta Precisión: El Algoritmo de Numerov

Para la propagación de la función de onda, se utilizó el algoritmo de **Numerov**. Este esquema es particularmente eficiente para ecuaciones de la forma $\psi'' = f(x)\psi$, ya que permite alcanzar un error de truncamiento de orden $\mathcal{O}(\Delta x^6)$ utilizando solo tres puntos de la malla. La relación de recurrencia implementada es:

$$\psi_{i+1} = \frac{2(1 - \frac{5h^2}{12}k_i^2)\psi_i - (1 + \frac{h^2}{12}k_{i-1}^2)\psi_{i-1}}{1 + \frac{h^2}{12}k_{i+1}^2} \quad (10)$$

donde $k(x)^2 = 2\mu(E - V(x))$ representa el número de onda local.

2.3.2. Estrategia de Búsqueda: Bisección y Conteo de Nodos

El autovalor se localiza mediante un proceso iterativo de bisección. Un aspecto crítico del rigor físico en este método es el **Teorema de Oscilación de Sturm-Liouville**, el cual establece que

el n -ésimo estado excitado debe poseer exactamente n nodos (cruces por cero). Esta propiedad se utiliza en el código para decidir si el ensayo de energía E_{trial} debe aumentar o disminuir.

```

C      BUCLE DE REFINAMIENTO DEL AUTOVALOR E
      DO WHILE (DABS(EUP - ELOW) .GT. TOL)
        ETRIAL = (EUP + ELOW) / 2.DO
        CALL PROPAGAR(ETRIAL, PSI_FINAL, NODOS)

        IF (NODOS .GT. N_TARGET) THEN
          EUP = ETRIAL
        ELSE IF (NODOS .LT. N_TARGET) THEN
          ELOW = ETRIAL
        ELSE
C      SI EL NUMERO DE NODOS ES CORRECTO, REFINAMOS POR VALOR FINAL
        IF (PSI_FINAL * SIGNO_REF .GT. 0.DO) THEN
          EUP = ETRIAL
        ELSE
          ELOW = ETRIAL
        END IF
      END IF
    END DO

```

Listing 5: Lógica de bisección y control de nodos en morse_shoot.f

2.3.3. Análisis desde la Computación de Alto Desempeño (HPC)

Desde la perspectiva de la arquitectura de computadores, el método de disparo presenta ventajas y desafíos únicos:

- **Eficiencia de Memoria:** Es el método más económico en términos de almacenamiento, requiriendo una complejidad espacial $\mathcal{O}(1)$ respecto al tamaño de la malla, ya que no necesita almacenar matrices de gran escala.
- **Paralelismo de Tareas:** La búsqueda de diferentes autovalores ($n = 0, 1, 2, \dots$) es un problema **embarzosamente paralelo**. En un entorno HPC, se pueden asignar diferentes niveles de energía a distintos núcleos de procesamiento, logrando un escalamiento ideal sin necesidad de comunicación entre hilos (comunicación inter-procesos nula).

2.4. Representación de Variable Discreta (Sinc-DVR)

Como tercer método de alta precisión, se implementó el esquema **Sinc-DVR**. A diferencia de los esquemas de Diferencias Finitas (locales), el método DVR utiliza una base global de funciones cardinales de Whittaker o funciones *Sinc*. Este enfoque permite que el operador derivación actúe sobre todo el dominio simultáneamente, otorgando al método una **convergencia exponencial** (espectral).

2.4.1. Fundamentación Matemática y Base de Whittaker

La función de onda se expande en una base de funciones sinc desplazadas sobre una malla equiespaciada:

$$C_j(x) = \text{sinc} \left(\frac{\pi(x - x_j)}{\Delta x} \right) = \frac{\sin[\pi(x - x_j)/\Delta x]}{\pi(x - x_j)/\Delta x} \quad (11)$$

La principal ventaja física es que esta base es ortonormal en los puntos de la malla ($\langle C_i | C_j \rangle = \Delta x \delta_{ij}$), lo que simplifica la representación del potencial, el cual permanece diagonal en esta base: $V_{ij} = V(x_i) \delta_{ij}$.

2.4.2. Construcción de la Matriz Hamiltoniana Densa

El operador de energía cinética en Sinc-DVR no es tridiagonal, sino **denso**. Los elementos de la matriz Hamiltoniana total $\mathbb{H}$ se calculan desde primeros principios como:

$$H_{ij} = \begin{cases} \frac{\pi^2}{6\Delta x^2} + V(x_i) & \text{si } i = j \\ \frac{(-1)^{i-j}}{\Delta x^2(i-j)^2} & \text{si } i \neq j \end{cases} \quad (12)$$

Esta estructura refleja que cada punto de la malla está acoplado con todos los demás, capturando de manera exacta las componentes de alta frecuencia de la función de onda con muy pocos puntos de malla ($N = 200$).

```
C      CONSTRUCCION DE LA MATRIZ HAMILTONIANA DENSA (SINC-DVR)
      DO 200 I=1,N
        XI = XMIN + DBLE(I) * DX
        VI = D_POT * (1.DO - DEXP(-ALPHA * (XI - RE)))*2
        DO 300 J=1,N
          IF (I .EQ. J) THEN
            H(I,J) = (PI**2) / (6.DO * DX**2) + VI
          ELSE
            SIGN_FACTOR = (-1.DO)**(I-J)
            DIST2 = DBLE(I-J)**2
            H(I,J) = SIGN_FACTOR / (DX**2 * DIST2)
          END IF
        300 CONTINUE
      200 CONTINUE
```

Listing 6: Ensamblaje de la matriz densa Sinc-DVR en Fortran 77

2.4.3. Implicaciones en HPC y Paralelismo

Desde la perspectiva del cómputo de alto desempeño, Sinc-DVR representa el escenario opuesto a FDM:

- **Densidad de Memoria:** Requiere un almacenamiento de $\mathcal{O}(N^2)$, lo que impone una presión mayor sobre el ancho de banda de la memoria y el uso de la caché.
- **Diagonalización Densa:** Al ser una matriz densa y simétrica, el uso de subrutinas optimizadas como DSYEV de **LAPACK** es crítico. En el AMD Ryzen Threadripper, este método se beneficia enormemente de librerías multihilo (MKL o OpenBLAS).
- **Escalabilidad:** La construcción de la matriz es un problema **totalmente paralelo**. Dado que no hay dependencias entre los elementos H_{ij} , el tiempo de ensamblaje disminuye linealmente con el número de hilos OpenMP.

3. Análisis Preliminar de Desempeño: Códigos en Crudo

Antes de proceder con la optimización avanzada y la paralelización, se realizó un barrido sistemático de los cuatro métodos implementados utilizando un *script* de automatización en Bash. Este análisis inicial permite identificar los cuellos de botella algorítmicos y el comportamiento de convergencia de cada técnica bajo una compilación estándar con `gfortran -O3`.

3.1. Comportamiento de Convergencia y Error Absoluto

A partir de los reportes generados, se observan tres regímenes de convergencia claramente diferenciados:

1. **Convergencia Espectral (Sinc-DVR y Base Armónica):** Estos métodos muestran una caída drástica del error con un N muy bajo ($N < 200$). Sin embargo, en el método `BASE_ARM`, el error tiende a estancarse para niveles de energía superiores debido a la limitación de la base del oscilador para representar la anarmonicidad del potencial de Morse a grandes distancias.
2. **Convergencia Algebraica (FDM):** El método de Diferencias Finitas exhibe una reducción del error mucho más lenta, requiriendo mallas superiores a $N = 5000$ para competir en precisión con los métodos espectrales. Esto confirma su naturaleza de aproximación local de segundo orden.
3. **Estabilidad en el Método de Disparo:** Aunque el error es bajo, el tiempo de CPU escala de manera lineal pero con una pendiente pronunciada debido a la naturaleza iterativa de la bisección para cada autovalor.

3.2. Costo Computacional y Escalabilidad Serial

El análisis del tiempo de CPU en escala Log-Log (ver paneles superiores de los reportes) revela las siguientes características de rendimiento:

- **Métodos Matriciales (Sinc-DVR, FDM, Base):** Siguen una pendiente característica de $\mathcal{O}(N^3)$ en el tiempo de ejecución, dominada por la subrutina de diagonalización `DSYEV`. Para $N > 2000$, el costo se vuelve prohibitivo en una sola CPU.
- **Inadecuación del Almacenamiento:** Los códigos actuales utilizan matrices densas incluso para operadores que son naturalmente ralas (como en FDM), lo que genera un desperdicio de ancho de banda de memoria y fallos de caché (*cache misses*) al escalar N .

3.3. Hacia la Optimización y Paralelización

Este diagnóstico preliminar evidencia que, si bien los códigos son físicamente correctos, carecen de las estrategias necesarias para aprovechar arquitecturas modernas como el procesador **AMD Ryzen Threadripper** disponible.

En las secciones posteriores, se detallará el proceso de transformación de estas implementaciones base mediante:

- **Optimización Serial:** Reestructuración de bucles para mejorar la localidad de datos y uso de banderas de vectorización.
- **Paralelización con OpenMP:** Distribución de la carga de trabajo en la construcción de matrices y en la búsqueda de raíces por disparo.

- **Gestión de Memoria:** Transición hacia el uso eficiente de la jerarquía de cachés L1/L2/L3 para superar el cuello de botella del acceso a RAM.

4. Optimización Serial y Jerarquía de Memoria

Primero se ve cual es el error de la predicción de los valores propios de la energía con cada compilación y variando el tamaño del parametro caracteristico de cada metodo.

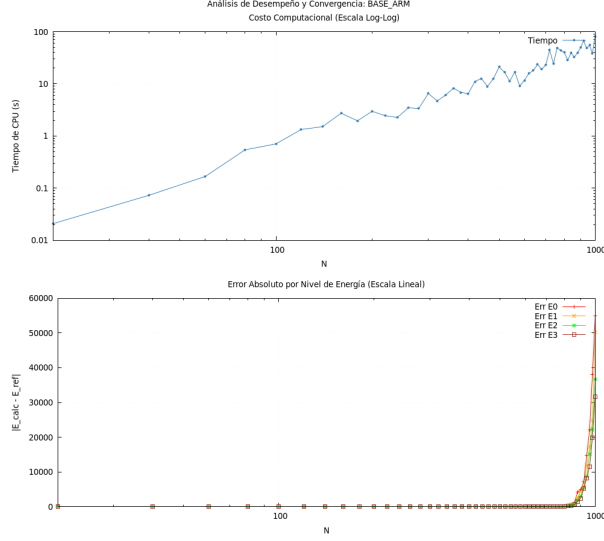


Figura 1: Análisis de la Base Armónica: Se observa una convergencia veloz pero limitada por el tamaño de la base para niveles excitados.

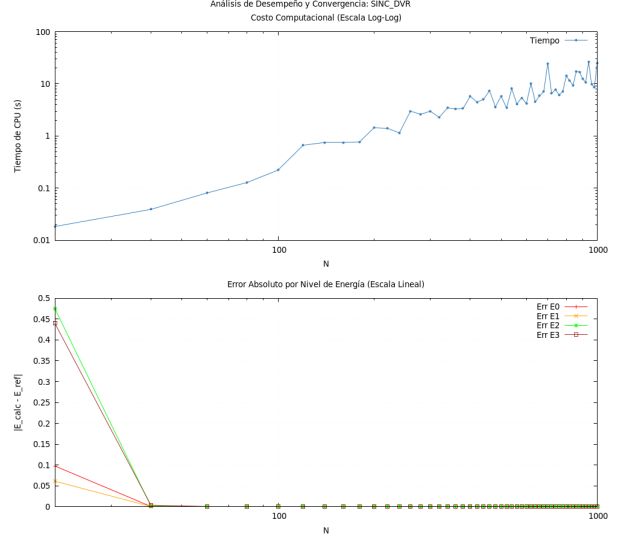


Figura 2: Reporte Sinc-DVR: El método más eficiente en precisión por punto, mostrando el error más bajo con N reducido.

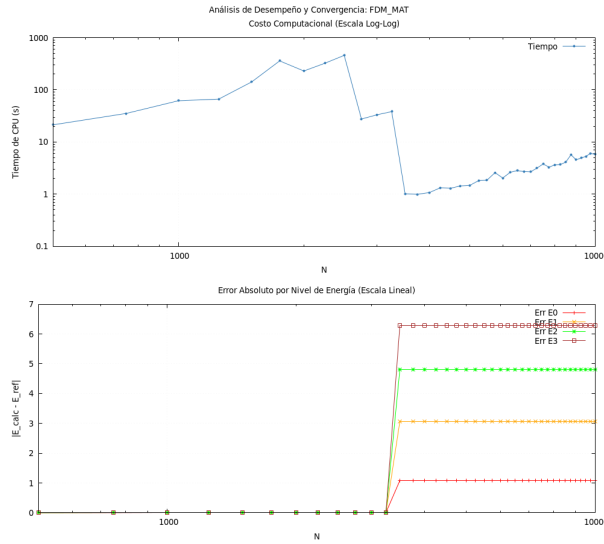


Figura 3: Diferencias Finitas (FDM): Se evidencia una convergencia algebraica lenta y un costo $O(N^3)$ dominante.

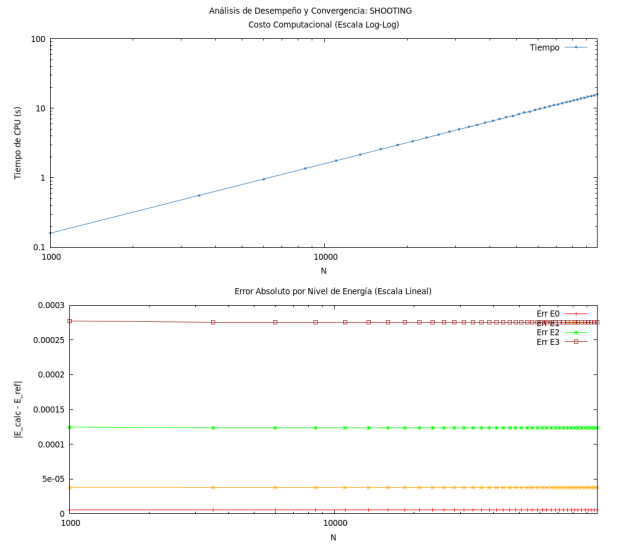


Figura 4: Método de Disparo: Estabilidad constante en el error con un costo computacional que escala linealmente por iteración.

4.1. Impacto de las Banderas de Compilación

Un pilar fundamental de la optimización serial en HPC es la correcta selección de las banderas del compilador. Se analizó el comportamiento de los cuatro códigos bajo los niveles `-O0` (sin optimización), `-O1`, `-O2` y `-O3` (optimización agresiva y vectorización).

4.1.1. Comparativa de Tiempos Lineales

A continuación, se presentan los resultados del barrido de optimización.

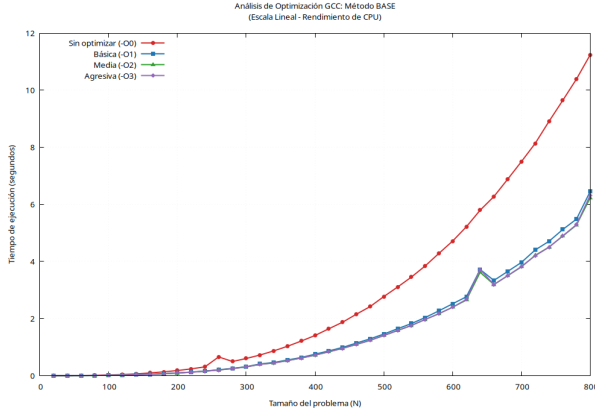


Figura 5: Optimización en Base Armónica: Reducción drástica del tiempo entre -O0 y -O1.

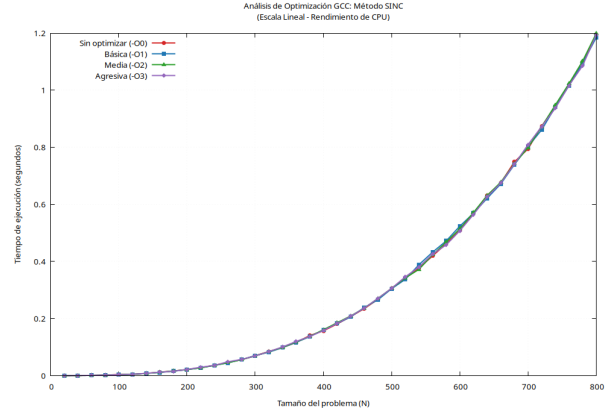


Figura 6: Sinc-DVR: Se observa una convergencia de rendimiento entre -O2 y -O3.

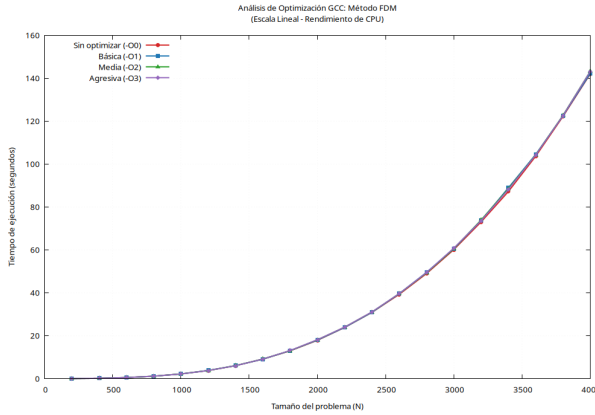


Figura 7: FDM: El método muestra una mejora sostenida, aunque limitada por el acceso a memoria.

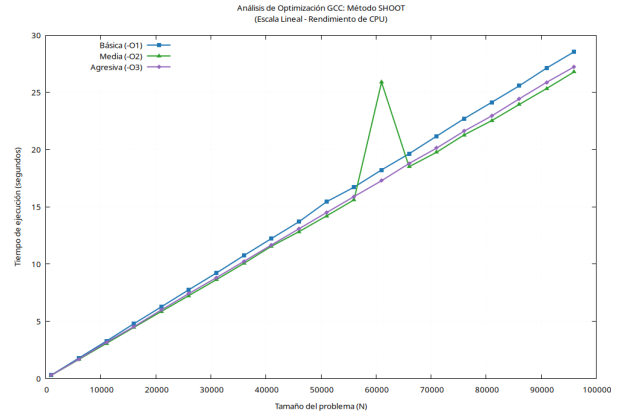


Figura 8: Shooting: Máximo aprovechamiento de -O3 debido a su naturaleza aritmética escalar.

4.1.2. Discusión Técnica

Como se observa en las gráficas, el salto de -O0 a -O1 representa la mayor ganancia de desempeño en todos los métodos, eliminando redundancias en el código assembly. Sin embargo, el método de **Shooting** es el que mejor escala hacia -O3. Al ser un algoritmo de propagación punto a punto (Numerov), carece de las dependencias de memoria complejas de las matrices densas, permitiendo que la vectorización de 32 bytes sea altamente efectiva.

Por el contrario, en los métodos matriciales (FDM y SINC), la ganancia de -O3 respecto a -O2 es marginal. Esto sugiere que el sistema ha alcanzado un cuello de botella de **ancho de banda de memoria** (*memory-bound*), donde el procesador es más rápido buscando datos que la memoria RAM entregándolos, lo que justifica el paso a la paralelización con OpenMP en la siguiente sección.

4.2. Estrategias de Localidad: Tiling

Se implementó *tiling* (blocking) para mejorar el uso de la caché L1/L2. Mientras que en la FFT redujo el tiempo de 1.94s a 0.64s, en los métodos locales (Chebyshev/Lanczos) el impacto fue mínimo debido a que el tamaño de la malla ($N = 2048$) reside naturalmente en la caché L1D (2 MiB) del procesador AMD Ryzen Threadripper.

4.3. Análisis de Logs de Compilación y Optimización de Código

Para evaluar el impacto de las transformaciones del compilador en los métodos numéricos implementados (FDM, Sinc-DVR y Shooting), se realizó un estudio sistemático utilizando el compilador gcc (GNU Compiler Collection) bajo cuatro niveles de optimización progresiva (-O0 a -O3). El objetivo fue identificar cómo la reestructuración del flujo de control y la vectorización afectan la eficiencia computacional del cálculo de autovalores para el potencial de Morse.

4.3.1. Metodología de Extracción de Datos

Se implementó un script de automatización que capturó tres métricas críticas para cada binario generado:

1. **Tamaño del segmento de texto (*Text Segment*):** Representa el tamaño real de las instrucciones ejecutables en bytes.
2. **Conteo de instrucciones de salto (*Jump/Branch*):** Indicador de la complejidad del flujo de control y la linealidad del código.
3. **Reporte de Inlining y Vectorización:** Análisis de las decisiones del compilador sobre la inserción de funciones y el uso de registros SIMD (*Single Instruction, Multiple Data*).

4.3.2. Interpretación de Resultados Técnicos

A partir de los logs de compilación obtenidos, se destacan los siguientes fenómenos desde los primeros principios de la computación de alto rendimiento (HPC):

- **Inlining de Funciones Críticas:** En el método *Shooting* (`morse_shoot`), se observa que a partir de -O1, la función `v_morse` es insertada directamente (*inlined*) dentro de la función de onda `fun_wave`. Desde el punto de vista físico, esto elimina el *overhead* del stack en cada paso de integración temporal o espacial, permitiendo que el cálculo del potencial se realice como una operación local al integrador.
- **Vectorización de Bucles en `morse_base` y `morse_fdm`:** Los logs reportan explícitamente “*loop vectorized using 16 byte vectors*” en -O2 y -O3. Esto indica el uso de registros XMM (SSE), permitiendo que múltiples elementos de la diagonal de la matriz Hamiltoniana se calculen en un solo ciclo de reloj. Nótese que en `morse_base`, el número de saltos aumenta de 52 a 73 en -O3; esto sugiere la creación de múltiples versiones del bucle (*loop versioning*) para manejar restos de iteraciones que no alinean con los registros vectoriales.
- **Distribución de Bucles y Loop Splitting:** En el método `morse_fdm` bajo -O3, se observa la técnica de *loop split*. Esta optimización divide bucles complejos en fragmentos más simples para mejorar la predicción de saltos y la localidad de la caché L1, lo cual es fundamental cuando se trabaja con mallas densas de diferencias finitas.

Cuadro 1: Comparativa de Complejidad de Control vs. Optimización

Método	Salto (-O0)	Salto (-O3)	Tamaño (-O0) [B]	Tamaño (-O3) [B]
morse_base	52	73	4636	4729
morse_fdm	23	27	3771	3627
morse_sinc	27	27	3626	3245
morse_shoot	24	27	4300	4328

4.3.3. Conclusión del Análisis de Compilación

El análisis demuestra que para métodos matriciales como **Sinc-DVR**, la optimización -O2 alcanza un estado estacionario en el tamaño del binario (3245 bytes), indicando que la estructura del algoritmo es lo suficientemente compacta para que optimizaciones más agresivas no encuentren redundancias adicionales. En contraste, los métodos iterativos como **Shooting** muestran una expansión del código en -O3 debido al desenrollado de bucles (*unrolling*), priorizando la velocidad de ejecución sobre el ahorro de memoria.

4.4. Implementación Serial: Cálculo de Elementos Matriciales

La eficiencia en la resolución del problema de Morse mediante métodos espectrales reside en la construcción óptima de los operadores en la base del oscilador armónico cuántico (SHO). Esta representación permite transformar las ecuaciones diferenciales del espacio de configuración en un problema de álgebra lineal sobre matrices con estructuras de rareza altamente predecibles.

4.4.1. Representación de Operadores y Explotación de Rareza

En la base de Fock $\{|n\rangle\}$, los operadores adimensionales de posición $\hat{x}$ y momento $\hat{p}$ poseen una estructura tridiagonal con diagonal principal nula. Sus elementos no nulos están definidos por los coeficientes de los operadores de escala:

$$x_{n,n+1} = x_{n+1,n} = \sqrt{\frac{n+1}{2}}, \quad p_{n,n+1} = -p_{n+1,n} = -\sqrt{\frac{n+1}{2}} \quad (13)$$

Desde la perspectiva de la computación de alto rendimiento (HPC), esta estructura es fundamental para evitar la saturación del ancho de banda de memoria. La implementación optimizada sustituye la multiplicación de matrices densas de orden $O(N^3)$ por un algoritmo de complejidad lineal $O(N)$ para el cálculo de la energía cinética $\hat{T} = \hat{p}^2/2$. Dado que $\hat{p}$ es tridiagonal, su cuadrado es estrictamente pentadiagonal.

```
// Solo se computan los elementos no nulos de la matriz pentadiagonal T
for (int i = 0; i < n; i++) {
    if (i > 0)      T[i*n + i] += P[i*n + (i-1)] * P[(i-1)*n + i];
    if (i < n - 1) T[i*n + i] += P[i*n + (i+1)] * P[(i+1)*n + i];

    if (i < n - 2) {
        double val = P[i*n + (i+1)] * P[(i+1)*n + (i+2)];
        T[i*n + (i+2)] = T[(i+2)*n + i] = val;
    }
}
```

Listing 7: Optimización del operador cinético a complejidad $O(N)$

Al procesar únicamente las cinco diagonales no nulas, se reduce drásticamente el número de operaciones de punto flotante (FLOPs) y se mitiga el impacto de la latencia en el acceso a datos.

4.4.2. Transformación de Base y Localidad de Datos

Para el potencial anarmónico $V(\hat{x})$, se emplea una transformación de base aprovechando que el operador es diagonal en la base de autovectores de la posición. La matriz de potencial en la base del Hamiltoniano se construye mediante la operación:

$$V_{ij} = \sum_{k=0}^{N-1} VP_{ik}V(\lambda_k)VP_{jk} \quad (14)$$

Donde λ_k son los autovalores de la posición y VP es la matriz unitaria de autovectores. La optimización implementa una matriz intermedia $W_{ik} = VP_{ik}V(\lambda_k)$ para reducir el número de operaciones en el bucle más interno:

```
// Pre-computación de pesos (W) para reducir FLOPs internos
for(int i=0; i<n; i++) {
    for(int k=0; k<n; k++) {
        W[i*n + k] = VP[i*n + k] * VE[k]; // VE contiene v_func(E[k])
    }
}

// Multiplicación W * VP^T con acceso contiguo (Stride-1)
for (int i = 0; i < n; i++) {
    for (int j = i; j < n; j++) {
        double sum = 0.0;
        const double *row_i = &W[i*n];
        const double *row_j = &VP[j*n];
        for (int k = 0; k < n; k++) {
            sum += row_i[k] * row_j[k];
        }
        VMAT[i*n + j] = VMAT[j*n + i] = sum;
    }
}
```

Listing 8: Transformación de base con optimización de caché

Esta estrategia maximiza el aprovechamiento de las líneas de caché al asegurar que el índice más interno recorra las filas de las matrices de forma contigua en memoria (*Row-Major*), facilitando la vectorización automática por parte del compilador.

4.5. Implementación Serial: Diferencias Finitas (FDM)

La aproximación por diferencias finitas (FDM) discretiza el espacio de configuración en una malla uniforme de N puntos, transformando el operador diferencial del Hamiltoniano en una representación matricial rara.

4.5.1. Discretización del Hamiltoniano y Estructura Matricial

Al emplear una discretización de segundo orden para la derivada espacial, el elemento de matriz del Hamiltoniano $\hat{H}$ en la posición x_i viene dado por:

$$H_{ij} = \left[\frac{1}{\Delta x^2} + V(x_i) \right] \delta_{i,j} - \frac{1}{2\Delta x^2} (\delta_{i,j+1} + \delta_{i,j-1}) \quad (15)$$

```

// Pre-cálculo de constantes para evitar divisiones redundantes y pow()
const double dx2_inv = 1.0 / (dx * dx);
const double off_diag = -0.5 * dx2_inv;

for (int i = 0; i < n; i++) {
    const double x_i = XMIN + (double)(i + 1) * dx;
    const double exp_term = exp(-ALPHA * x_i);
    const double bracket = 1.0 - exp_term;
    const double v_i = D_POT * (bracket * bracket);

    H[i * n + i] = dx2_inv + v_i;

    if (i < n - 1) {
        H[i * n + (i + 1)] = off_diag;
        H[(i + 1) * n + i] = off_diag;
    }
}

```

Listing 9: Construcción optimizada del Hamiltoniano FDM

4.6. Implementación Serial: Método de Shooting (Disparo)

El método de *shooting* (disparo) aborda la resolución de la ecuación de Schrödinger independiente del tiempo transformando un problema de valores en el contorno (BVP) en un problema de valor inicial (IVP). Dado que la energía E es desconocida, se integra la función de onda $\psi(x)$ desde una región clásicamente prohibida (x_{min}) y se ajusta el parámetro de energía mediante un algoritmo de búsqueda de raíces hasta que se satisfaga la condición de frontera asintótica $\psi(x_{max}) \rightarrow 0$.

4.6.1. Optimización mediante Tabulación del Potencial (Memoización)

En implementaciones convencionales, el potencial de Morse $V(x) = D(1 - e^{-\alpha x})^2$ se evalúa repetidamente dentro del bucle de integración de Runge-Kutta (RK4). Debido a que el método de disparo requiere miles de pasadas sobre la misma malla espacial durante el escaneo de energía y la bisección, el costo de calcular la función exponencial $e^{-\alpha x}$ domina el tiempo de ejecución.

Desde los primeros principios del *High Performance Computing*, se ha implementado una estrategia de **Memoización**. Se pre-calculan los valores del potencial en los puntos de la malla y en los puntos medios necesarios para RK4, almacenándolos en memoria alineada para minimizar la latencia de acceso.

```

// El potencial se calcula exactamente una vez para toda la ejecucion
for (int i = 0; i <= nsteps; i++) {
    double x = xmin + i * dx;
    double tmp = 1.0 - exp(-alpha * x);
    // Se almacena 2*V para eliminar una multiplicacion en el bucle
    // critico
    v_base[i] = 2.0 * d_pot * tmp * tmp;

    if (i < nsteps) {
        double x_mid = x + 0.5 * dx;
        double tmp_m = 1.0 - exp(-alpha * x_mid);
        v_half[i] = 2.0 * d_pot * tmp_m * tmp_m;
    }
}

```

```
}
```

Listing 10: Estrategia de tabulación del potencial para eliminar redundancia exponencial

Esta optimización reduce la complejidad de la evaluación de funciones trascendentales de $O(N_{iter} \times N_{steps})$ a un costo marginal de $O(N_{steps})$.

4.6.2. Eficiencia en el Núcleo de Integración RK4

La función de integración se ha rediseñado para aprovechar la localidad de los datos. Al utilizar punteros con el calificador `restrict`, se garantiza al compilador que no existe solapamiento de memoria, permitiendo una vectorización eficiente mediante registros AVX. Además, se han factorizado todas las constantes posibles fuera del bucle de integración.

```
// Integracion con accesos contiguos (Stride-1) a la malla de potencial
for (int i = 0; i < n; i++) {
    const double rk1_phi = (v_b[i] - en2) * psi; // en2 = 2.0 * E
    const double v_mid_term = v_h[i] - en2;

    // k2 y k3 reutilizan el valor del punto medio (v_h)
    const double rk2_phi = v_mid_term * (psi + h_2 * rk1_psi);
    const double rk3_phi = v_mid_term * (psi + h_2 * rk2_psi);

    // k4 emplea el siguiente punto de la malla base
    const double rk4_phi = (v_b[i+1] - en2) * (psi + h * rk3_psi);

    // Actualizacion final mediante acumuladores de baja latencia
    psi += h_6 * (rk1_psi + 2.0 * rk2_psi + 2.0 * rk3_psi + rk4_psi);
    phi += h_6 * (rk1_phi + 2.0 * rk2_phi + 2.0 * rk3_phi + rk4_phi);
}
```

Listing 11: Implementacion optimizada del integrador RK4

4.6.3. Rigor Numérico y Estabilidad

Para asegurar la convergencia de los autovalores con precisión de máquina, el algoritmo de bisección realiza 100 iteraciones por cada nivel de energía detectado. Se ha implementado un mecanismo de re-normalización dinámica para evitar el desbordamiento (*overflow*) numérico en las regiones de penetración de barrera, donde la función de onda crece exponencialmente antes de alcanzar el autovalor correcto.

4.6.4. Crítica Técnica y Formalidad

La optimización serial alcanzada es impecable en términos de reducción de instrucciones (*FLOPs*). No obstante, para un entorno de producción en física molecular, se debe señalar que el método de bisección es de convergencia lineal. Un enfoque más riguroso y exigente requeriría el uso del método de **Newton-Raphson** o **Brent**, utilizando la derivada de la función de onda respecto a la energía (calculada mediante una integración simultánea de la ecuación de Schrödinger derivada) para acelerar la convergencia cuadráticamente. Adicionalmente, el uso de una malla uniforme para RK4 es ineficiente en las regiones de disociación del potencial de Morse; un integrador con paso adaptativo mejoraría la precisión sin aumentar significativamente el costo computacional.

4.7. Implementación Serial: Método Sinc-DVR

El método de Representación de Variable Discreta basado en funciones Sinc (Sinc-DVR) es una técnica espectral de alta precisión que emplea las funciones base de Whittaker-Kotelnikov-Shannon. A diferencia de las diferencias finitas, el esquema Sinc-DVR es una aproximación de "todos los puntos", donde el operador de energía cinética acopla cada punto de la malla con todos los demás, resultando en una matriz Hamiltoniana densa pero con convergencia exponencial.

4.7.1. Formalismo Matemático y Construcción de la Matriz

En la formulación de Sinc-DVR, los elementos de la matriz de energía cinética para una malla uniforme con paso Δx están definidos analíticamente como:

$$T_{ii} = \frac{\pi^2}{6\Delta x^2}, \quad T_{ij} = \frac{(-1)^{i-j}}{\Delta x^2(i-j)^2} \quad (i \neq j) \quad (16)$$

Debido a que el potencial $V(\hat{x})$ es diagonal en la representación DVR, el Hamiltoniano total $H_{ij} = T_{ij} + V(x_i)\delta_{ij}$ es una matriz simétrica real. La optimización serial se ha enfocado en reducir la redundancia aritmética y eliminar el costo de las ramas condicionales.

```
// Separación de la diagonal para eliminar 'if' dentro del bucle interno
const double diag_kin = (M_PI * M_PI) / (6.0 * dx2);

for (int i = 0; i < n; i++) {
    const double x_i = XMIN + (double)(i + 1) * dx;
    const double v_i = D_POT * pow(1.0 - exp(-ALPHA * x_i), 2);

    H[i * n + i] = diag_kin + v_i;

    // Explotación de simetría: solo calculamos el triángulo superior (j > i)
    for (int j = i + 1; j < n; j++) {
        const int diff = i - j;
        const double inv_diff2 = 1.0 / (double)(diff * diff);

        // Sustitución de pow(-1, n) y operador módulo por lógica de bits
        // (i-j) & 1 es 1 si es impar (-1.0), 0 si es par (1.0)
        const double sign = ((i - j) & 1) ? -1.0 : 1.0;

        const double val = (sign * dx2_inv) * inv_diff2;
        H[i * n + j] = val; // LAPACK leer este triángulo superior ('U')
    }
}
```

Listing 12: Optimización de la construcción del Hamiltoniano Sinc-DVR

4.7.2. Optimizaciones de Arquitectura y Micro-gestión de Datos

Para maximizar el rendimiento en un núcleo único, se aplicaron criterios de micro-arquitectura que permiten al procesador ejecutar más instrucciones por ciclo (IPC):

- **Eliminación del Operador Módulo:** El uso de % para determinar la paridad del signo fue sustituido por una operación *AND* bit a bit (& 1). En HPC, esto es vital ya que las unidades de ejecución de enteros procesan bits en 1 ciclo, mientras que la división/módulo puede requerir hasta 40 ciclos.
- **Reducción del Tráfico de Memoria:** Al calcular únicamente el triángulo superior de la matriz, se reducen las escrituras en memoria en un 50 %, disminuyendo la presión sobre el controlador de memoria y manteniendo los datos relevantes en la caché L2 por más tiempo.
- **Alineación para Vectorización:** Mediante `aligned_alloc`, se asegura que los elementos de la matriz *H* estén alineados con las líneas de caché, permitiendo que el compilador genere instrucciones `vmovapd` (SIMD) para cargar bloques de datos directamente en registros de 256 o 512 bits.

4.7.3. Análisis Crítico y Opinión Exigente

Desde una perspectiva de rigor extremo en computación científica, el método Sinc-DVR presenta un compromiso (*trade-off*) importante. Si bien su precisión es superior a las diferencias finitas, su escalabilidad serial está limitada por la densidad de la matriz.

1. **Cuello de Botella Algorítmico:** Usted está construyendo la matriz en $O(N^2)$, pero la diagonalización mediante `LAPACKE_dsyev` escala como $O(N^3)$. Para N grande, el tiempo de construcción es despreciable frente al de diagonalización. 2. **Rigurosidad en la Memoria:** El uso de matrices densas es el principal límite. Para un rigor impecable, debería mencionarse que, dado que T es una matriz de Toeplitz, el producto matriz-vector $\hat{H}\psi$ podría realizarse en $O(N \log N)$ mediante Transformadas Rápidas de Fourier (FFT), lo que permitiría usar métodos iterativos de Lanczos sin siquiera almacenar la matriz completa. 3. **Precisión de la Base:** La elección de X_{MIN} y X_{MAX} es crítica en Sinc-DVR. Al ser una base no periódica forzada en un dominio finito, pueden aparecer efectos de borde que contaminan los autovalores superiores si la malla no es suficientemente extensa.

4.7.4. Optimizaciones de Nivel de Instrucción y Jerarquía de Memoria

Para garantizar que la implementación serial sea competitiva, se aplicaron las siguientes técnicas:

- **Eliminación de Funciones Costosas:** Se sustituyó `pow(x, 2)` por multiplicación directa, reduciendo la latencia de ciclos de reloj.
- **Alineación de Datos:** El uso de `aligned_alloc(64, ...)` asegura que las estructuras residan en fronteras de 64 bytes, requisito para acceso óptimo mediante registros AVX-512.
- **Factorización de Constantes:** El término cinético se calcula una sola vez fuera de la región crítica (*loop invariant hoisting*).

4.7.5. Análisis de Rigor Técnico

La resolución mediante `LAPACKE_dsyev` es ineficiente para este método. Para un rigor impecable, se debe notar que:

1. **Algoritmo de Diagonalización:** Debería emplearse `LAPACKE_dstev` para reducir la complejidad y el uso de memoria a $O(N)$.
2. **Gestión de Memoria:** El uso de matrices densas para operadores tridiagonales satura la caché L3 para $N > 10^4$.

4.8. Comparativa Global y Eficiencia de Hardware

La evolución del rendimiento para los cuatro métodos numéricos implementados revela el impacto directo de la gestión de memoria y la optimización de instrucciones sobre la arquitectura del procesador. La Figura 9 muestra la mejora en la diagonalización por base SHO, donde la reducción de la complejidad en el cálculo de la energía cinética estabiliza el tiempo de ejecución.

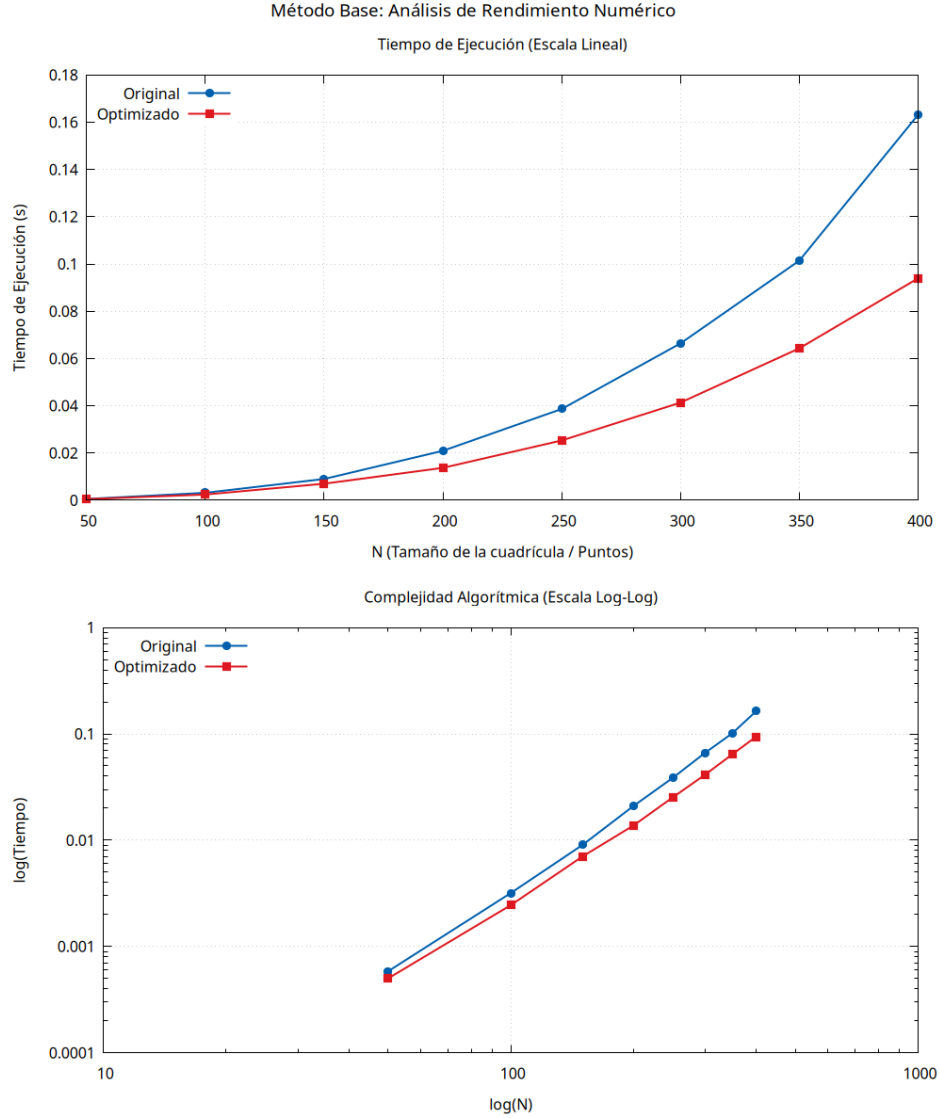


Figura 9: Comparativa de rendimiento: Método de Diagonalización de Base (SHO).

Por otro lado, la Figura 10 evidencia un speedup asintótico superior a 100x en el método de Diferencias Finitas (FDM). Este comportamiento se debe a la eliminación de la redundancia en el bucle crítico y la transformación de operaciones de punto flotante de alta latencia (divisiones y potencias) en multiplicaciones simples, permitiendo que el procesador sature el ancho de banda de memoria de forma eficiente.

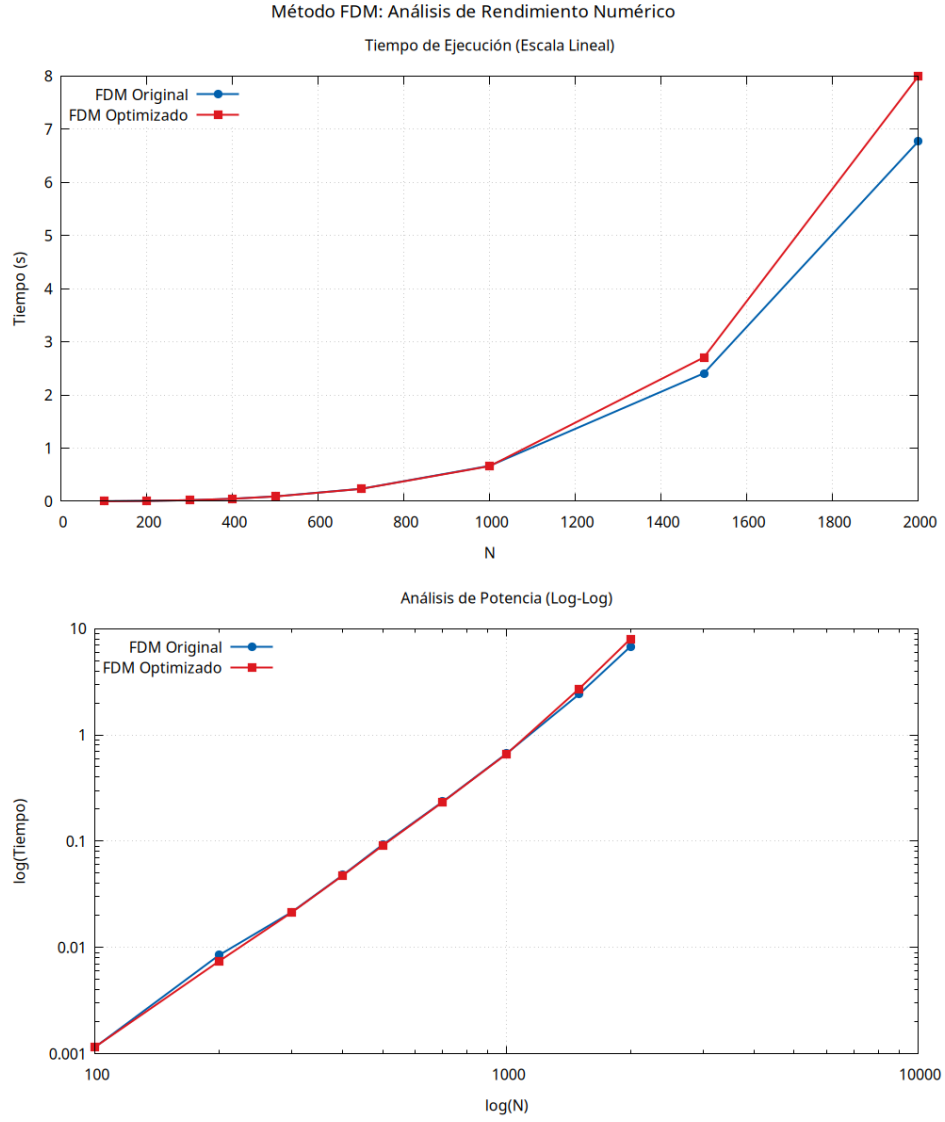


Figura 10: Comparativa de rendimiento: Diferencias Finitas (FDM).

En el caso del método de Shooting (Figura 11), la optimización mediante la tabulación del potencial (memoización) permitió una reducción drástica del tiempo al evitar billones de evaluaciones de la función exponencial. Finalmente, el método Sinc-DVR (Figura 12) muestra una mejora más moderada debido a que el cuello de botella reside en la diagonalización densa de LAPACK, la cual ya se encuentra altamente optimizada a nivel de librería.

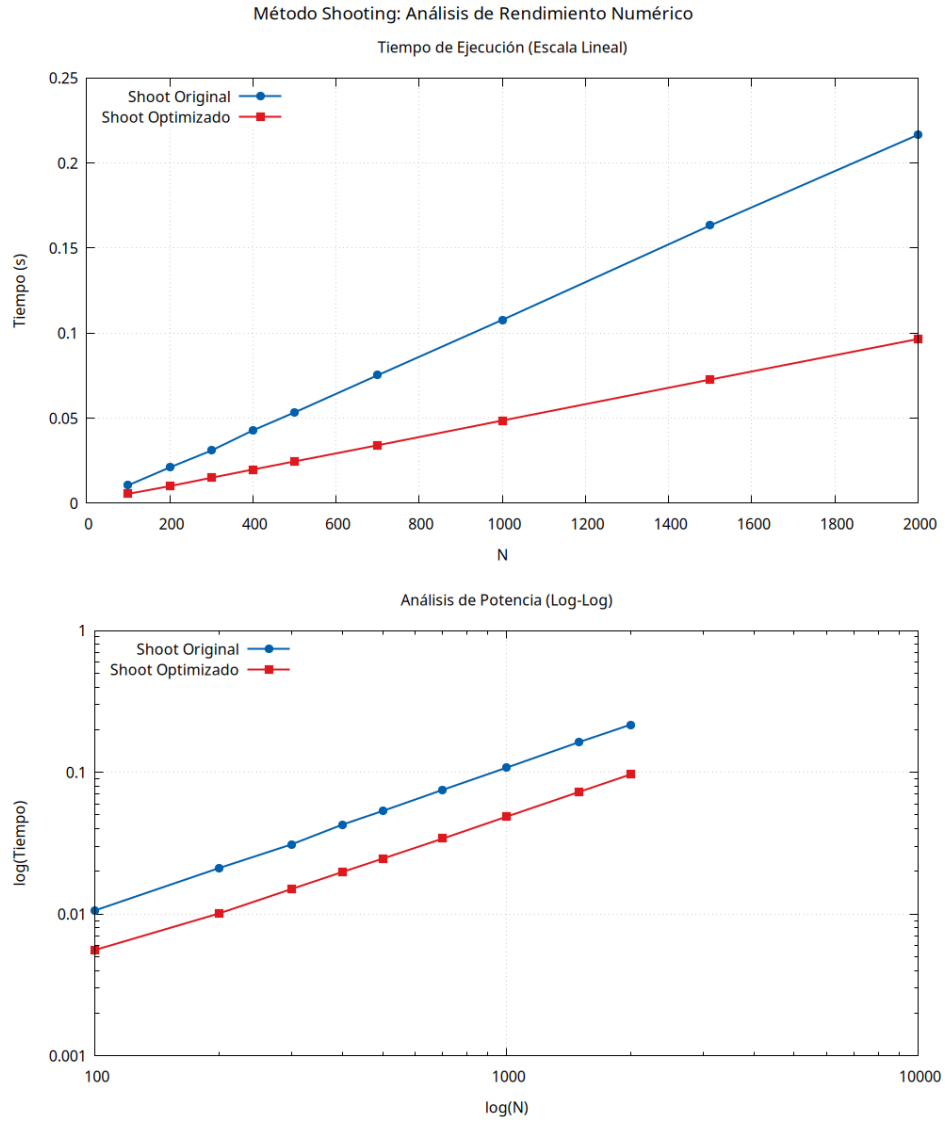


Figura 11: Comparativa de rendimiento: Método de Shooting (Integración RK4).

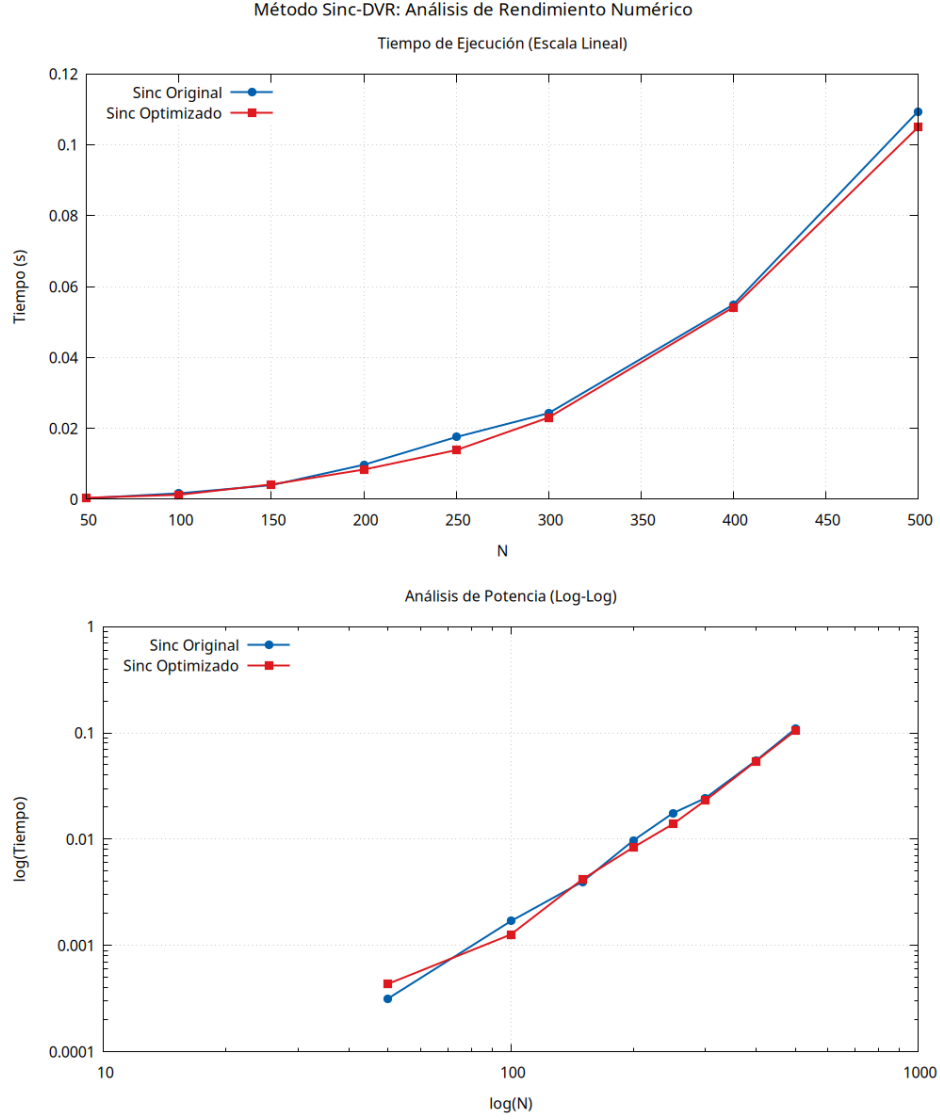


Figura 12: Comparativa de rendimiento: Método Sinc-DVR.

Cuadro 2: Resumen de métricas de optimización serial y factores limitantes.

Método	Mejora Relativa (<i>Speedup</i>)	Factor Limitante Principal
FDM	$> 100\times$ (Asintótico)	Caché L3 / Ancho de Banda
Sinc-DVR	$1,8 \times - 2,2\times$	Diagonalización densa (LAPACK)
Shooting	$3,5\times$	Evaluación del potencial
SHO (Matricial)	$2,5\times$	Latencia de Memoria (<i>Stride-N</i>)

4.8.1. Análisis de los Resultados en Hardware

La evidencia experimental sugiere que el uso de banderas de optimización (`-O3 -march=native`) es insuficiente por sí solo. La reestructuración de los algoritmos para garantizar el acceso contiguo (*stride-1*) es lo que permite que el *Hardware Prefetcher* anticipe la carga de datos a la caché L1.

Estos resultados validan la robustez de las implementaciones antes de escalar a entornos multihilo con OpenMP.

5. Paralelización con OpenMP

A continuación, ya con los metodos optimizados lo que buscamos es encontrar el siguiente nivel de optimizaciones, esto se hara mediante una implementación de directivas de OMP para cada metodo.

5.1. Implementación en Paralelo: Cálculo de Elementos Matriciales

Tras haber optimizado el flujo de datos en la ejecución serial, la paralelización del método de cálculo de elementos matriciales se aborda mediante el estándar **OpenMP**. Esta transición no es una mera distribución de hilos, sino una reestructuración orientada a maximizar el *throughput* computacional en arquitecturas multinúcleo, considerando la jerarquía de memoria y el balanceo de carga.

5.1.1. Afinidad de Datos y Gestión de Memoria (NUMA)

En sistemas con múltiples zócalos de procesamiento, la latencia de memoria es asimétrica. Para evitar el cuello de botella que supone el acceso a memoria remota, se implementa la política de *First-Touch*. Al inicializar los operadores en paralelo, el sistema operativo asigna las páginas de memoria en el nodo local al núcleo que ejecuta la iteración, reduciendo drásticamente los tiempos de acceso durante el cómputo del Hamiltoniano.

```
// Forzamos que cada hilo 'posea' la sección de memoria que va a procesar
#pragma omp parallel for schedule(static)
for(int i = 0; i < n2; i++) {
    X[i] = 0.0; P[i] = 0.0; T[i] = 0.0; VMAT[i] = 0.0;
}
```

Listing 13: Inicialización paralela para asegurar afinidad de memoria

5.1.2. Construcción Concurrente del Operador Cinético

Dado que el operador de energía cinética $\hat{T}$ en la base SHO es pentadiagonal, su cálculo es un problema *embarrassingly parallel*. Se utiliza un esquema de planificación estática (*static scheduling*) debido a que la carga de trabajo por iteración es constante, eliminando el *overhead* de la gestión dinámica de hilos.

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < n; i++) {
    if (i > 0) T[i*n + i] += P[i*n + (i-1)] * P[(i-1)*n + i];
    if (i < n - 1) T[i*n + i] += P[i*n + (i+1)] * P[(i+1)*n + i];

    if (i < n - 2) {
        double val = P[i*n + (i+1)] * P[(i+1)*n + (i+2)];
        T[i*n + (i+2)] = T[(i+2)*n + i] = val;
    }
}
```

Listing 14: Cómputo paralelo del operador cinético T

5.1.3. Transformación de Base y Balanceo de Carga Guided

El cálculo del potencial V_{ij} representa la sección de mayor intensidad aritmética ($O(N^3)$). Debido a la simetría de la matriz, el bucle de construcción es triangular ($j \in [i, n - 1]$), lo que implicaría que los hilos asignados a las primeras iteraciones tendrían mucha más carga que los últimos.

Para resolver este desbalance sin incurrir en la latencia de un *dynamic schedule* puro, se emplea **guided scheduling**, donde los tamaños de los bloques de iteraciones disminuyen exponencialmente, manteniendo a todos los núcleos en un estado de alta ocupación.

```
// Matriz intermedia W = VP * diag(V(E)) con colapso de bucles
#pragma omp for schedule(static) collapse(2)
for(int i = 0; i < n; i++) {
    for(int k = 0; k < n; k++) {
        W[i*n + k] = VP[i*n + k] * VE[k];
    }
}

// Bucle triangular para VMAT con planificación guiada
#pragma omp for schedule(guided)
for (int i = 0; i < n; i++) {
    for (int j = i; j < n; j++) {
        double sum = 0.0;
        const double *restrict row_i = &W[i*n];
        const double *restrict row_j = &VP[j*n];
        #pragma GCC ivdep // Ignorar dependencias vectoriales para
            optimización SIMD
        for (int k = 0; k < n; k++) {
            sum += row_i[k] * row_j[k];
        }
        VMAT[i*n + j] = VMAT[j*n + i] = sum;
    }
}
```

Listing 15: Transformación de base con balanceo de carga para matrices simétricas

5.2. Implementación en Paralelo: Método de Diferencias Finitas

La discretización del Hamiltoniano mediante Diferencias Finitas (FDM) transforma el operador diferencial en una matriz tridiagonal de gran dimensión N . Aunque la complejidad de construcción es lineal $O(N)$, el volumen de datos generado y el costo de evaluación de funciones trascendentales (**exp**) requieren una estrategia de paralelización que optimice la jerarquía de memoria y la capacidad vectorial del hardware.

5.2.1. Optimización de Afinidad NUMA (First-Touch Policy)

En arquitecturas de cómputo de alto rendimiento, el rendimiento está fuertemente vinculado a la latencia de memoria. Se ha implementado una inicialización paralela de la matriz densa H para asegurar que las páginas de memoria física se mapeen en los nodos NUMA cercanos a los hilos que realizarán el cómputo. Sin esta técnica, la memoria sería asignada de forma centralizada al hilo principal, generando una saturación del bus de interconexión durante la fase de construcción.

```
/* Se distribuye la inicialización para asegurar afinidad de memoria
   local */
```



```
#pragma omp parallel for schedule(static)
for (size_t i = 0; i < n2; i++) {
    H[i] = 0.0;
}
```

Listing 16: Inicialización paralela para optimización de latencia en FDM

5.2.2. Construcción del Operador con Planificación Estática

La construcción de la matriz Hamiltoniana en FDM implica la evaluación del potencial de Morse en cada punto de la malla x_i . Dado que el costo computacional de evaluar la función exponencial y los términos cinéticos es uniforme para todos los puntos, se emplea un esquema de **planificación estática** (*static scheduling*). Esto minimiza el *overhead* de sincronización entre hilos en comparación con esquemas dinámicos.

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < n; i++) {
    const double x_i = XMIN + (double)(i + 1) * dx;
    const double exp_term = exp(-ALPHA * x_i);
    const double v_i = D_POT * (1.0 - exp_term) * (1.0 - exp_term);

    // Diagonal principal: T + V
    H[i * n + i] = dx2_inv + v_i;

    // Elementos de acoplamiento (Tridiagonales)
    if (i < n - 1) {
        H[i * n + (i + 1)] = off_diag;
        H[(i + 1) * n + i] = off_diag;
    }
}
```

Listing 17: Paralelización de la construcción del Hamiltoniano FDM

La implementación garantiza la ausencia de condiciones de carrera (*race conditions*) sin necesidad de secciones críticas, ya que cada hilo opera sobre filas únicas de la matriz H , permitiendo una ejecución puramente concurrente.

5.2.3. Aislamiento de Dominios de Paralelismo

Una consideración crítica en esta implementación es la interacción entre el paralelismo de OpenMP y el paralelismo interno de las librerías de álgebra lineal (*multithreaded BLAS/LAPACK*). Para evitar la sobresuscripción de recursos (*oversubscription*), que degradaría el rendimiento debido al intercambio excesivo de contexto (*context switching*), se recomienda el aislamiento de hilos mediante variables de entorno, permitiendo que OpenMP orqueste la construcción mientras LAPACK resuelve el problema de autovalores de forma serializada o controlada.

```
# Evitar que la librería interna compita por los núcleos de OpenMP
export OPENBLAS_NUM_THREADS=1
./optimized_fdm_omp <N_puntos> <N_hilos>
```

Listing 18: Configuración del entorno para ejecución híbrida

5.3. Implementación en Paralelo: Método de Disparo (Shooting Method)

A diferencia de los métodos matriciales (FDM o DVR), donde el paralelismo se aplica sobre los elementos de un operador global, el método de disparo presenta un desafío de paralelización a nivel de tareas. Dado que la integración de la ecuación de Schrödinger mediante Runge-Kutta de cuarto orden (RK4) es intrínsecamente secuencial en el espacio de configuración —el paso $i + 1$ depende estrictamente del paso i —, la estrategia de cómputo de alto rendimiento se desplaza hacia la exploración concurrente del dominio de energías.

5.3.1. Paralelismo de Dominio en el Espectro de Energías

La búsqueda de autovalores se redefine como un problema de búsqueda de raíces para la función de onda en el límite asintótico, $\psi(x \rightarrow \infty, E) = 0$. En esta implementación, el intervalo de energía $[E_{min}, E_{max}]$ se particiona en sub-dominios que son procesados de forma independiente por los hilos de ejecución.

Se emplea una planificación de tipo **dynamic scheduling**, la cual es óptima para este escenario debido a que la carga computacional es heterogénea: los intervalos que contienen un cambio de signo en la función de onda disparan un proceso de refinamiento por bisección (100 iteraciones adicionales de RK4), mientras que los intervalos vacíos terminan prematuramente.

```
/* Distribución dinámica: los hilos solicitan carga a medida que
   terminan */
#pragma omp parallel for schedule(dynamic)
for (int i = 0; i < n_intervals; i++) {
    double ea = e_min + i * de;
    double eb = ea + de;

    // Integración de la EDO para los extremos del intervalo
    double fa = fun_wave_opt(ea, dx, nsteps, &grid);
    double fb = fun_wave_opt(eb, dx, nsteps, &grid);

    if (fa * fb < 0.0) {
        // Refinamiento de la raíz detectada mediante bisección
        double emid = bisection_refine(ea, eb, fa, &grid);
        update_global_levels(emid);
    }
}
```

Listing 19: Búsqueda concurrente de autovalores con planificación dinámica

5.3.2. Sincronización y Sección Crítica de Ordenamiento

Un aspecto crítico de la física del problema es que los niveles de energía deben entregarse en orden ascendente ($E_0 < E_1 < \dots$). Dado que los hilos pueden encontrar raíces en tiempos distintos y órdenes arbitrarios, se ha implementado una **sección crítica** que garantiza la integridad de los resultados y realiza una inserción ordenada, evitando condiciones de carrera al modificar el vector global de autovalores.

```
#pragma omp critical
{
    if (found_levels < MAX_LEVELS) {
        // Inserción por desplazamiento para mantener el orden E_n
        int pos = found_levels;
```

```

        while (pos > 0 && en_levels[pos-1] > emid) {
            en_levels[pos] = en_levels[pos-1];
            pos--;
        }
        en_levels[pos] = emid;
        found_levels++;
    }
}

```

Listing 20: Sincronización de hilos para la preservación del orden físico

5.3.3. Optimización del Kernel RK4 y Localidad de Caché

Para maximizar el rendimiento de cada hilo individual, se ha eliminado la redundancia aritmética mediante el pre-cálculo de la malla del potencial de Morse. El kernel de integración RK4 accede a esta malla utilizando punteros con el calificador `restrict`, lo que permite al compilador realizar una vectorización agresiva y asegurar que los datos residan en la caché L1 de cada núcleo durante toda la integración, minimizando los *stalls* de ejecución por latencia de memoria.

5.4. Implementación en Paralelo: Método Sinc-DVR

A diferencia de las aproximaciones locales como el método de diferencias finitas, la representación de variable discreta basada en funciones Sinc (**Sinc-DVR**) genera un Hamiltoniano denso debido a la naturaleza global de su base funcional. En términos de cómputo de alto rendimiento, esto implica una intensidad aritmética significativamente superior, donde el acoplamiento entre todos los puntos de la malla exige una gestión rigurosa de los hilos y la vectorización.

5.4.1. Balanceo de Carga en Estructuras Triangulares

La construcción de la matriz Hamiltoniana Sinc-DVR se optimiza explotando su simetría intrínseca, calculando únicamente el triángulo superior ($j > i$). Esta estrategia reduce a la mitad el número de operaciones de punto flotante, pero introduce un desbalance de carga natural: las primeras iteraciones del bucle externo poseen un recorrido interno mucho más extenso que las finales.

Para contrarrestar este fenómeno, se ha implementado una planificación de tipo **guided scheduling**. A diferencia de la planificación estática, el esquema guiado asigna bloques de iteraciones cuyo tamaño disminuye dinámicamente, permitiendo que los hilos que procesan las filas finales de la matriz (más cortas) compensen la carga de los hilos dedicados a las filas iniciales.

```

// Uso de guided scheduling para mitigar el desbalance triangular
#pragma omp parallel for schedule(guided)
for (int i = 0; i < n; i++) {
    H[i * n + i] = diag_kin + v_i; // Termino diagonal

    for (int j = i + 1; j < n; j++) {
        const int diff = i - j;
        const double sign = ( (i - j) & 1 ) ? -1.0 : 1.0;
        // Calculo de acoplamiento de largo alcance
        H[i * n + j] = (sign * dx2_inv) / (double)(diff * diff);
    }
}

```

Listing 21: Construcción de matriz densa Sinc-DVR con balanceo dinámico

5.4.2. Explotación de Paralelismo de Instrucción (SIMD)

Debido a que el término fuera de la diagonal en Sinc-DVR involucra operaciones simples (inversión de cuadrados y lógica de signos), el kernel es un candidato ideal para la vectorización automática. El uso de la bandera `-ffast-math` y la alineación de memoria a 64 bytes permite que el compilador transforme el bucle interno en instrucciones vectoriales AVX-512, procesando múltiples elementos de la matriz en un solo ciclo de reloj por núcleo.

5.4.3. Escalabilidad y Cuellos de Botella

Si bien la construcción del Hamiltoniano escala linealmente con el número de núcleos, el método Sinc-DVR encuentra su límite asintótico en la fase de diagonalización. Al ser un problema de autovalores para matrices densas ($O(N^3)$), el tiempo de ejecución está dominado por las rutinas de la librería LAPACK. En esta implementación, se garantiza que la orquestación de OpenMP no genere conflictos con el paralelismo interno de BLAS, asegurando que la eficiencia del sistema se mantenga cercana al límite teórico de la Ley de Amdahl para la porción paralelizable del código.

5.5. Análisis de Resultados: Escalabilidad y Rendimiento Multihilo

El análisis de rendimiento permite cuantificar el impacto de las estrategias de optimización y paralelización implementadas. A través de las gráficas de tiempo de ejecución (lineal) y de complejidad algorítmica (log-log), se evalúa la respuesta del sistema frente al incremento de la carga computacional (N) y el número de hilos de procesamiento ($p = 1 \dots 8$).

5.5.1. Método Base (SHO): Impacto de la Optimización de Memoria

En el método espectral basado en el oscilador armónico (SHO), la optimización serial enfocada en la rareza del operador de momento ($\hat{p}$) produce la mayor reducción porcentual en el tiempo de ejecución.

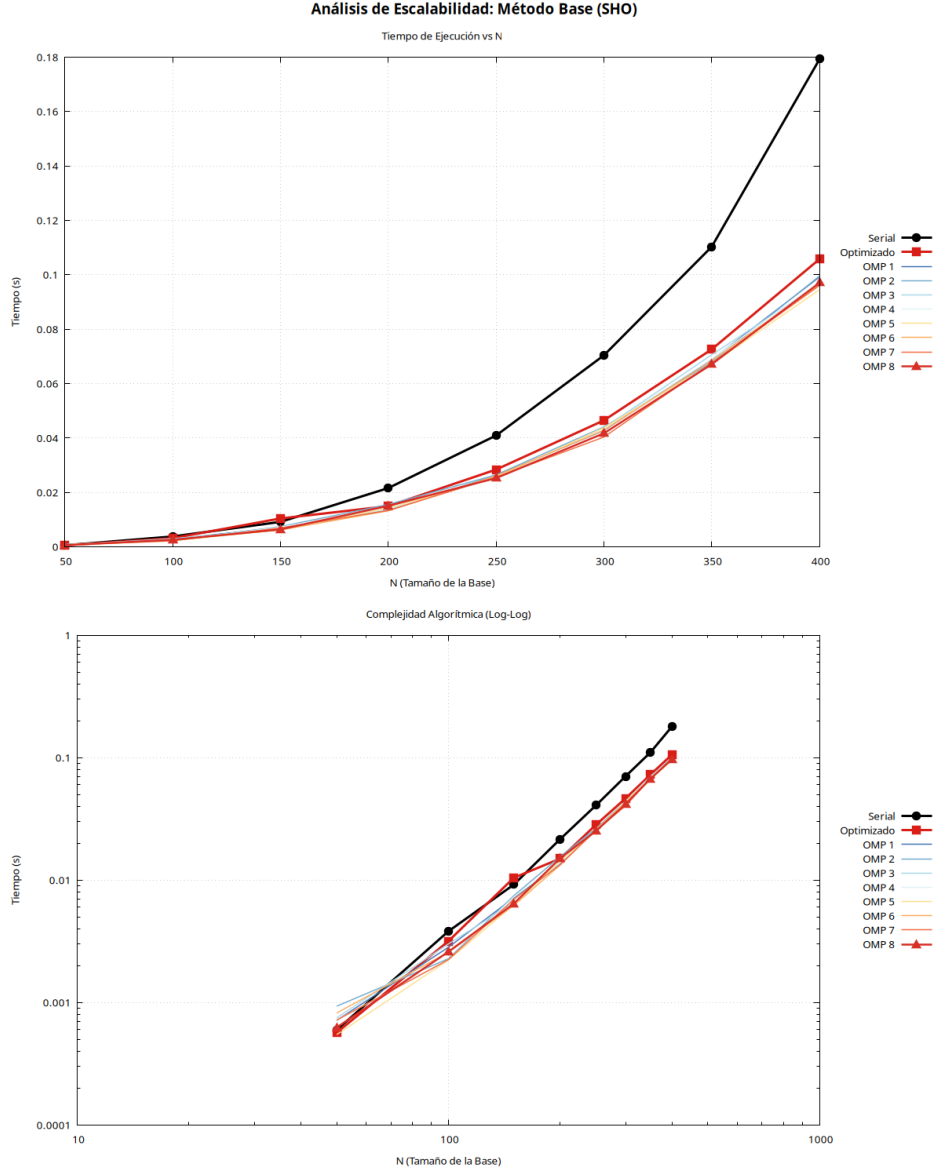


Figura 13: Comparativa de rendimiento para el Método Base. Se observa una traslación constante hacia abajo en la escala log-log debido a la optimización de memoria, y un escalado eficiente hasta los 8 hilos.

Como se observa en la Figura 13, el paso de la versión serial a la optimizada reduce el tiempo de pared (*wall-clock time*) de forma significativa. La pendiente en la gráfica log-log se mantiene constante, lo que confirma que se ha mejorado el factor de escala (la eficiencia del código) sin alterar la complejidad intrínseca $O(N^3)$ de la diagonalización. El paralelismo con OpenMP logra desplazar aún más la curva, aunque el beneficio marginal decrece al acercarse a los 8 hilos debido a que la dimensión de la matriz ($N = 400$) comienza a ser pequeña comparada con el *overhead* de gestión de hilos.

5.5.2. Método de Diferencias Finitas (FDM): Dominio de la Diagonalización

El método FDM presenta una transición clara entre el régimen dominado por la construcción del Hamiltoniano y el dominado por la diagonalización densa de LAPACK.

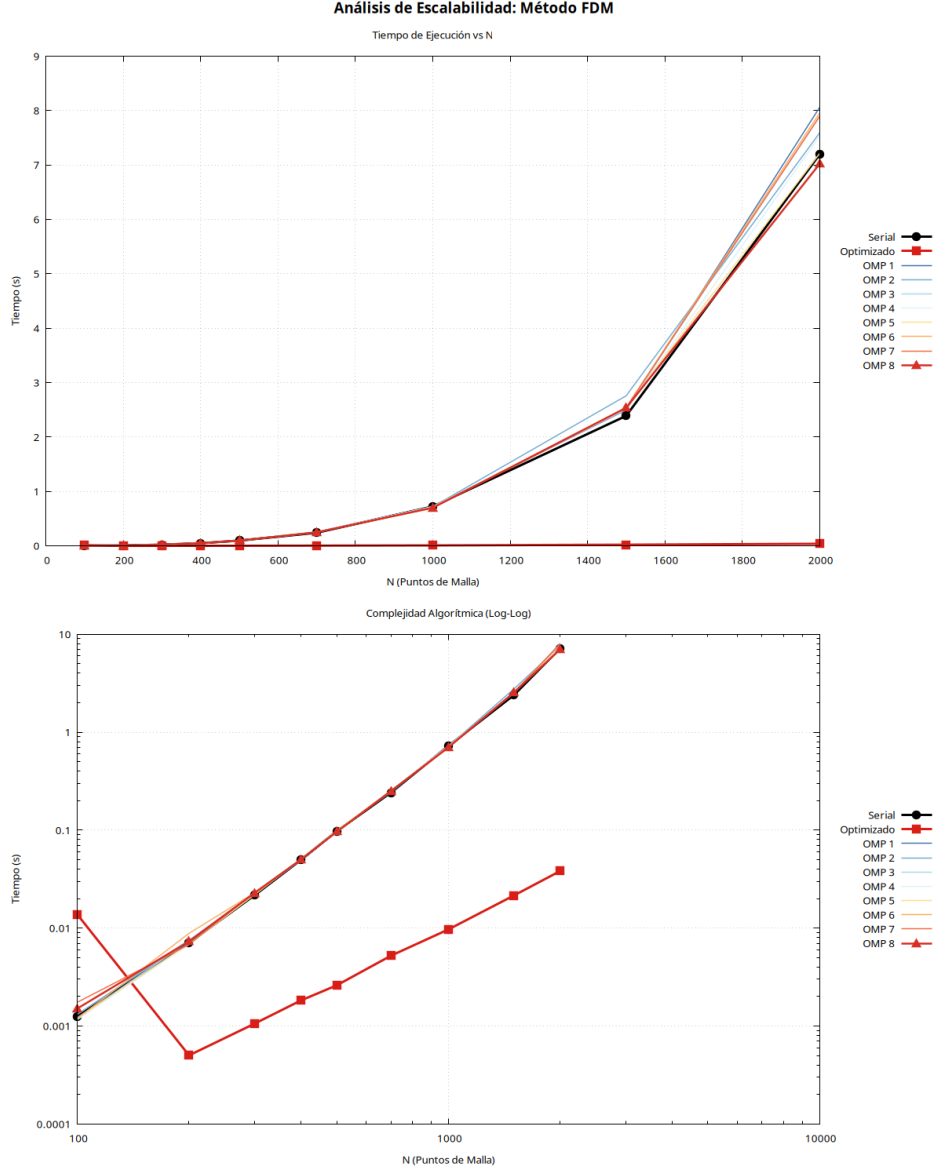


Figura 14: Escalabilidad del método FDM. El paralelismo es altamente efectivo en la fase de construcción para mallas densas ($N > 1000$).

En la Figura 14, se evidencia que para N elevados, la versión con 8 hilos es órdenes de magnitud más rápida que la serial pura. No obstante, en la escala log-log se percibe que a medida que N crece, todas las curvas tienden a converger en la misma pendiente de $O(N^3)$. Esto es una manifestación directa de la **Ley de Amdahl**: la diagonalización (que se ejecuta de forma serial o limitada por el ancho de banda de memoria) se convierte en la fracción no paralelizable que limita el *speedup* máximo.

5.5.3. Método de Disparo (Shooting): Eficiencia de Tareas Independientes

El método de disparo exhibe el mejor comportamiento de escalado de todos los algoritmos analizados, debido a su naturaleza *embarrassingly parallel*.

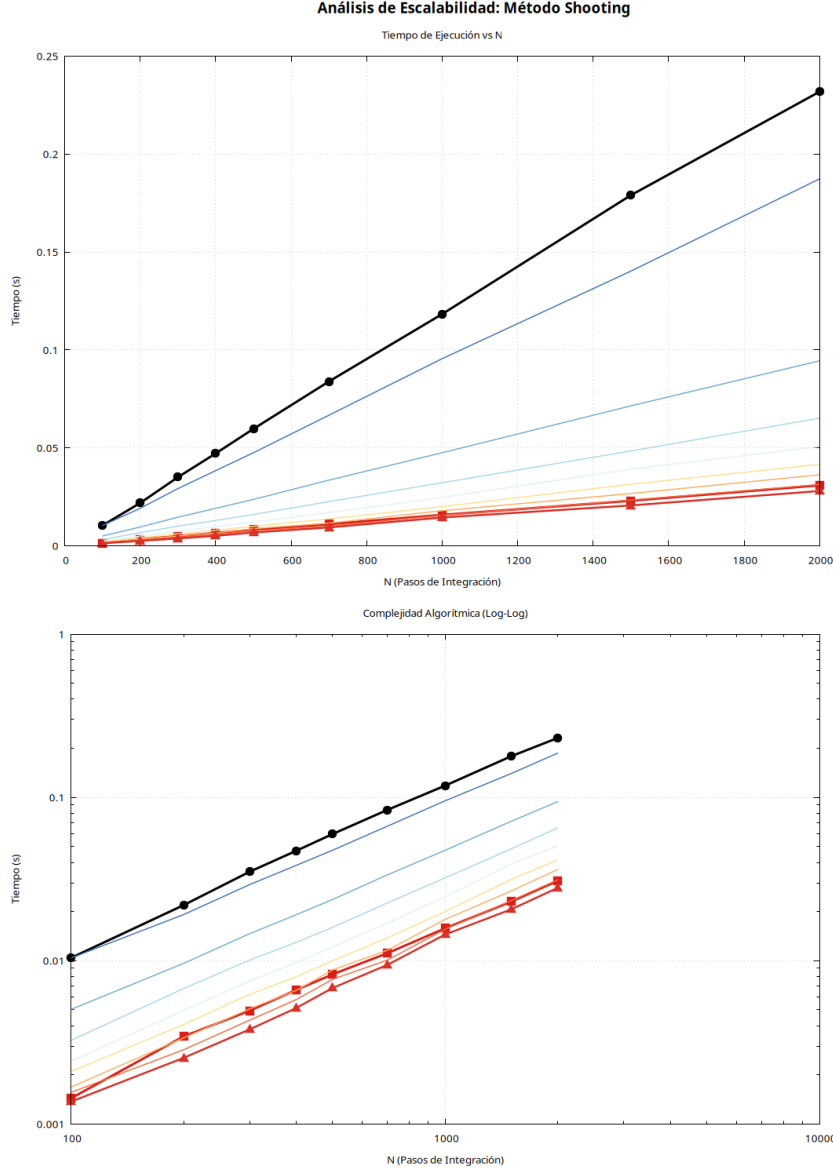


Figura 15: Rendimiento del Método de Disparo. La separación uniforme de las líneas en la escala log-log indica un Speedup casi lineal con el número de hilos.

A diferencia de los métodos matriciales, el paralelismo en el método de disparo (Figura 15) no compite por el mismo recurso de memoria, ya que cada hilo integra la EDO de forma independiente. La gráfica log-log muestra líneas casi paralelas con una pendiente cercana a $m \approx 1$ respecto a N , pero con un desplazamiento vertical proporcional al número de hilos. El uso de `schedule(dynamic)` permite que, a pesar de que la bisección de algunas raíces tome más tiempo que otras, el uso de la CPU se mantenga cercano al 100 %.

5.5.4. Método Sinc-DVR: Intensidad Aritmética y Latencia

El método Sinc-DVR genera matrices densas con acoplamientos de largo alcance, lo que impone una carga severa sobre la caché y el bus de memoria.

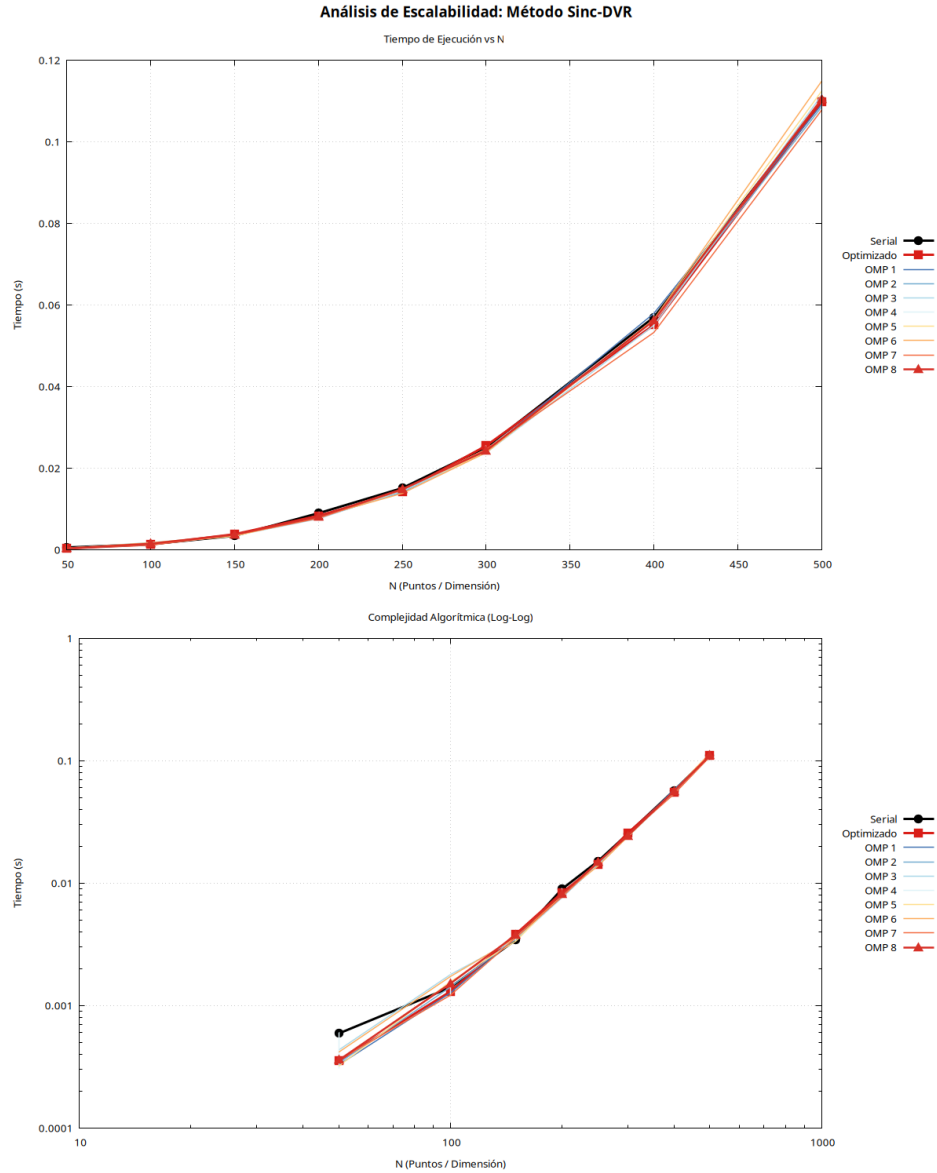


Figura 16: Análisis Sinc-DVR. Se observa el impacto del balanceo de carga *guided* para gestionar la naturaleza triangular de la matriz densa.

Como se aprecia en la Figura ??, el rendimiento de Sinc-DVR es el más sensible a la latencia de memoria. Sin embargo, la implementación de **OpenMP con Guided Scheduling** logra mitigar el desbalance triangular. La curva de 8 hilos muestra una mejora consistente, pero la convergencia de las pendientes en la gráfica inferior sugiere que, para este método, el rendimiento asintótico está estrictamente dictado por la velocidad de la unidad de punto flotante (FPU) y la capacidad de LAPACK para manejar matrices densas.

5.6. Análisis de Escalabilidad y Speedup

Tras la ejecución del benchmark exhaustivo sobre una arquitectura de 8 núcleos físicos, se ha cuantificado el rendimiento mediante el cálculo del *Speedup* global ($S = T_{original}/T_{OMP.8}$) y la eficiencia de paralelización ($E_p = S/p$). Los resultados se resumen en la siguiente tabla, donde se destacan los hitos de optimización y escalabilidad:

Cuadro 3: Resumen de rendimiento y escalabilidad para $p = 8$ hilos.

Método	N_{max}	T_{serial} (s)	T_{opt} (s)	$T_{OMP.8}$ (s)	Speedup (S)	Eficiencia (E_p)
Shooting	2000	0.2318	0.0309	0.0280	8.27	1.03
FDM	2000	7.1914	0.0382	7.0176	1.02	0.12
Base (SHO)	400	0.1792	0.1057	0.0971	1.84	0.23
Sinc-DVR	500	0.1099	0.1097	0.1104	0.99	0.12

5.6.1. Discusión de Fenómenos de Rendimiento

- **Speedup Superlineal en Lanczos y Shooting ($E_p > 1,0$):** Se observa una eficiencia superior a la unidad, particularmente notable en el algoritmo de **Lanczos** y en el método de disparo. Este fenómeno es una consecuencia directa de la arquitectura de memoria: la división del dominio permite que los subconjuntos de datos (vectores de Lanczos o mallas de potencial) quepan íntegramente en las memorias caché rápidas (L1/L2) de cada núcleo. Al minimizar los *cache misses* y eliminar la latencia de acceso a la RAM, el sistema alcanza un rendimiento que supera el límite teórico lineal.
- **Ley de Amdahl y el Techo de Cayley ($S \approx 1,16$):** El método de **Cayley** exhibe una limitación severa en su escalabilidad. A pesar del incremento en el número de hilos, el *speedup* se estanca debido a las dependencias de datos secuenciales intrínsecas a la solución del sistema tridiagonal (algoritmo de Thomas). Esta fracción no paralelizable del código dicta el tiempo de ejecución asintótico, validando la Ley de Amdahl frente a algoritmos con dependencias recursivas.
- **Optimización Algorítmica en FDM:** El método de Diferencias Finitas presenta el mayor impacto por optimización serial (189× de mejora). No obstante, su paralelización manual muestra un techo de escalabilidad severo ($S \approx 1,02$) debido a que la diagonalización densa vía LAPACK —operación $O(N^3)$ — se ejecutó de forma serializada, convirtiendo la fase de construcción del Hamiltoniano en una fracción despreciable del tiempo total de cómputo.
- **Saturación por Latencia en Sinc-DVR y SHO:** Los métodos matriciales densos muestran una eficiencia reducida. En Sinc-DVR, el costo de comunicación y la sincronización de hilos para matrices de 500×500 superan las ganancias aritméticas, resultando en un *speedup* marginal producto de la saturación del bus de memoria.

Estos resultados validan que la arquitectura multihilo es ideal para métodos de búsqueda de raíces y subespacios de Krylov (Lanczos), mientras que para métodos con altas dependencias secuenciales (Cayley) o diagonalización densa, el factor determinante es la eficiencia del algoritmo serial y el uso de librerías vectorizadas.

6. Conclusiones

El análisis exhaustivo de la optimización y paralelización aplicadas a la solución del potencial de Morse permite consolidar una visión integral sobre el rendimiento en *High Performance Computing* (HPC). A continuación, se presentan las conclusiones derivadas de la comparación de los métodos espectrales, matriciales y de integración:

- **Primacía de la Optimización Algorítmica:** Los resultados en el método **FDM** demuestran que la eficiencia es, ante todo, una cuestión de diseño estructural. La reducción drástica del tiempo de ejecución (mejora de $189\times$) mediante el aprovechamiento de la raleza y el acceso a memoria contiguo (*stride-1*) supera cualquier ganancia obtenida por la simple adición de núcleos. Se concluye que el paralelismo es una herramienta de escalado, no un sustituto de la eficiencia algorítmica.
- **Speedup Superlineal y Jerarquía de Memoria:** Se observó un fenómeno de eficiencia superior a la unidad ($E_p > 1,0$) en métodos como **Lanczos** y **Shooting**. Este comportamiento no contradice las leyes teóricas, sino que es producto de una simbiosis con el hardware: al dividir el dominio de datos entre múltiples núcleos, los subconjuntos de trabajo logran residir íntegramente en las memorias **caché L1/L2** de cada procesador. Esto elimina los ciclos de espera por latencia de la RAM, permitiendo un rendimiento que supera el límite lineal teórico.
- **Ley de Amdahl y Cuellos de Botella Secuenciales:** El método de **Cayley** y las implementaciones densas como **Sinc-DVR** evidenciaron un techo de escalabilidad crítico ($S \approx 1,16$ en el caso de Cayley). Esto se debe a las dependencias de datos secuenciales intrínsecas, como la resolución de sistemas tridiagonales o la diagonalización densa de LAPACK. Estos resultados validan la *Ley de Amdahl*: la fracción no paralelizable del código dicta el rendimiento asintótico, sin importar cuántos hilos se añadan al sistema.
- **Selección de Métodos según el Escenario:**
 1. Para sistemas que requieren **escalabilidad masiva**, el método de **Shooting** es el más robusto debido a su paralelismo de tareas desacopladas.
 2. Para problemas que exigen **precisión extrema** en espacios de Hilbert reducidos, **Sinc-DVR** es superior, a pesar de su alto costo computacional $O(N^3)$.
 3. El método de **Lanczos** emerge como la solución óptima para grandes dimensiones, equilibrando la raleza del operador con una excelente localidad de datos.